Scotland Yard report

ss24495 & er24668

Throughout the coursework, we have produced a working build of the code that has passed all of the internal tests provided. We only made a slight attempt on the CW-AI part of the coursework, completing only a basic MrX score function, as we felt that the time investment it would have required would not have brought an equivalent in marks given for it. Spending a greater amount of time on the CW-MODEL aspect was a wiser decision.

Our paired approach to this coursework was to coordinate what subclasses and methods we were focusing on in the work to each other. This was to make sure that neither of us pushed a git commit and either overwrote what the other person has written or spend a long time having to comb through a merging request in the code, outlining every line of code that was to be deleted and what was to be kept. When it got to parts of the coursework that were more challenging, we focused on them with the 2- person 1-computer approach in the dedicated labs sessions, proving to be helpful in both of us giving our inputs into an approach and covering for each other's mishaps that may have occurred.

One issue occurred from this when implementing the advance and move methods during the coding process. One of us had implemented partway through the advance method while the other was starting the singleMoves method. When working through singleMoves, advance had to be altered because the logic was incorrect and did not allow singleMoves to be correctly made. The person who originally started on advance forgot to git pull before starting again so when it came to committing their work, a huge portion of the code was receiving overwrite requests and it had to be all selected as to which parts were kept and removed. This was a setback in the work, but further emphasised the importance of communication between each other and making sure that your code was fully up to date by using git fetch and git pull, before starting any work as to prevent any accidental branching.

As to our understanding, the game works off of a basis where each game state is immutable and therefore aspects of each game state cannot be changed or altered when created. This means that for every move or instance where the game would have to change in some way, a new game instance must be created and implemented to reflect any in-game changes. This was different to previous games that we may have created, but this approach keeps each representation of the game state extremely well defined and robust. By keeping aspects such as the moves, pieces remaining and log entries as immutable, it meant that these could not be altered by accident by stray code.

It also means that any intermediate states that are created – for example in doubleMoves – are not generated into the game until it is sure that it was a legal operation. This ideology demonstrates a focus on safety and ease of reasoning within the code, avoids accidental bugs and ensures a constant state.

A big achievement to us was our implementation of advance and the visitor pattern. By 'visiting' the move with Move.Visitor, the visitor pattern allows the program to handle each type of move separately from the move itself, and gives the option of handling both single

and double moves differently, cleanly separating them and keeping the visitor maintainable. It eliminates the need of explicit casting and showcases a use of encapsulation. The most challenging aspect of implementing advance was being able to access the current player, since there was no easily defined variable for this. After looking deeply into the game's structure we found that we could use remaining to determine the current player indirectly. Since the game was initiated with remaining empty, we used an empty remaining to identify the current player as MrX. Then when advanced was called during MrX's turn we added the detectives in the game to remaining to signify that it would be the detectives turn for the new game state. We later realised we needed to add the extra condition of only adding detectives that had tickets for the implementation of getWinner to function correctly.

By extension, another strong achievement was the makeSingleMove and makeDoubleMove helper methods for getAvailableMoves. We found the large majority of makeDoubleMoves quite easy to implement after makeSingleMove as we used the same concept of iterating each adjacent node and checking if the player has the required tickets, but with another set of nested for loops to find the second move with the first move destination as the new source. However it was initially challenging to pass most tests as there were more edge cases to consider with double moves, but analysing the error messages and using print statements in our code helped us identify what we were missing. To check if the node was unoccupied by a detective, we at first tried to iteratively check every pieces location against the nodes found and then return whether the node was empty using a flag system. We were hesitant to create a break statement into our code but we couldn't think of a solution that wouldn't require us rewriting a lot of our previous logic so we decided to code it in anyway. This is certainly a part of the code that we would have come up with a better solution if we had more time.

A critical point of our code that was hard to code was the observer interface. We had to wrap the game state in a model object and then return any state changes accurately to observers. We initially passed the first few tests, but then had big issues with one test that wouldn't return correctly to the observer. Our model kept returning an incorrect winner to the observer, one of them specifically when RED was supposed to be the winner and MRX was returned, or vice versa. We kept trying to alter our logic to the MyModelFactory code to try and remedy this issue, but to no avail. Ultimately, the issue was our winner logic within MyGameStateFactory. Although it passed all the tests, the logic was slightly incorrect and we had to alter it to provide a correct result to the observer. By removing some conditional statements that were producing erroneous results and introducing the winner statement of winner being defined as a stream of the players filtered to not MRX if a detective lands on MRX's piece, we were able to fix this issue and pass all of the tests.

A limitation of our coursework is the lack of most of the AI implementation into the code, especially since we took the time to understand downcasting, its limitations, and ways to avoid it. This proved to be because of time constraints and we didn't think it was worth our time based on how much we spent starting it. I think our time management was a factor to blame for this, as I think we underestimated the task we had to complete, as well not enough of our time coding spent physically together. If we were to do this project again, I think we would start understanding and planning the project as soon as possible to make its implement easiest, and arrange for more in-person sessions as it did feel limiting coding individually for a larger than-necessary proportion of the project.